Mathematical Formalization of Neural Networks (MLP) from First Principles

Gennadii Iarkov

March 2026

Contents

Introduction

Artificial neural networks are currently one of the most effective and fast growing machine learning tools. Due to their ability to model complex non-linear data, they find vast usage in various fields, from image detection to creation of Large Language Models (LLM). A classical neural network architecture is the Multilayer Perceptron (MLP).

This work presents the mathematical theory and foundations of how the multilayer perceptron functions and "learns". As a universal formalization does not exist, this work is based on various sources to construct a cohesive definitional framework of its own.

The first section focuses on the base theory of artificial neural networks. It formulates the concepts of the neuron, activation function, as well as defines the architecture of a multilayer perceptron. The second section concentrates on the "learning" process of the multilayer perceptron with the aim of optimizing its parameters. It defines the concepts of cost and loss, describes the algorithm of backpropagation and the iterative method of gradient descent. The third section includes the theory regarding the problem of overfitting of neural networks, where a dropout regularization method is described.

The theory regarding a singular neuron, activation functions, MLP architecture and dropout regularization are mainly based on the manual of Goodfellow *et al.* [3] as well as the publication of H.Kinsley and D.Kukieła [2]. The mathematical description of the "learning" process of the multilayer perceptron, specifically the backpropagation algorithm, is mainly based on the publication of M.A. Nielsen [1].

1 The Architecture of a Neural Network

1.1 Singular Neuron

Firstly we formulate the concept of a neural network, namely an MLP (Multilayer Perceptron) type neural network. The smallest unit of a multilayer perceptron is an artificial neuron. It can be thought of as a mathematical function, which transforms an n-dimensional input vector into a scalar output value.

Each element of the input vector is "connected" with the neuron by a *weight*. Additionally, each neuron contains the so called *bias parameter*. To start with, we introduce the definition of an *activation function*:

Definition 1. *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, differentiable almost everywhere function. Then σ is called an activation function.*

Reasons, for which the function must be defined this way, will become evident in the further sections. Next, we define the singular neuron:

Definition 2. *Let $x \in \mathbb{R}^n$ be an input vector, $w \in \mathbb{R}^n$ a weight vector and $b \in \mathbb{R}$ a bias parameter. The function $f : \mathbb{R}^n \to \mathbb{R}$ is then called an artificial neuron and is given by the formula:*

$$f(x) = \sigma(w^T x + b),$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ is an activation function.

The activation function is a key element of the artificial neuron. A simple affine transformation does not allow for the modeling of non-linear relationships in data. The introduction of a non-linear activation function σ allows the neural network to approximate every continuous function (Universal Approximation Theorem).

There exist many different activation functions. One of them is the sigmoid function $\sigma : \mathbb{R} \to (0, 1)$ defined as:

$$\sigma(x) = \frac{1}{(1 + e^{-x})}.$$

The sigmoid function however possesses certain drawbacks when used in neural networks (issues of vanishing gradients, computational inefficiency, etc.) In practice, the ReLU (Rectified Linear Unit) function is most commonly used.

Definition 3. *The function $\sigma : \mathbb{R} \to [0, \infty)$ is called the ReLU function and is defined as:*

$$\sigma(x) = \max(0, x).$$

It can be noted, that it's a non-linear function that is differentiable almost everywhere (where $x \neq 0$). Calculating the derivative of the ReLU function is also very simple, especially compared to the sigmoid function.

1.2 Neuron Layers

Single neurons usually aren't capable of solving complex problems. All the input information would have to be condensed into a single neuron, which would lead to massive information loss. We can then add more parameters to the model, by introducing a great number of neurons. In that way we construct the so called *layer*.

Remark 1.1 (Pointwise application of the activation function). Because the activation function σ is defined for scalar arguments, we adopt the convention of pointwise application to use the function on vectors. For a vector $x = (x_1, x_2, ..., x_n)^T \in \mathbb{R}^n$, the activation function σ acts by applying that function on all subsequent vector elements:

$$\sigma(x) = \begin{bmatrix} \sigma(x_1) \\ \sigma(x_2) \\ \vdots \\ \sigma(x_n) \end{bmatrix} \in \mathbb{R}^n.$$

Definition 4. *Let $x \in \mathbb{R}^m$ be an input vector. A neural network layer, consisting of n neurons is defined as a mapping $f : \mathbb{R}^m \to \mathbb{R}^n$ given by the formula:*

$$f(x) = \sigma(Wx + b),$$

where W is a $n \times m$ weight matrix, $b \in \mathbb{R}^n$ is the vector of biases of each neuron in the layer, and σ is an activation function applied element-wise on the resulting vector.

Before we move onto the final definition of a multilayer perceptron, we first introduce some terminology:

- **Input Layer:** the vector containing the input data. Can be adapted to fit the prior definition of a neural network layer if necessary, however in this work, it will be treated as a vector of input data.

- **Hidden Layer:** the neural network layer located "in between" other layers that performs non-linear transformations with an activation function.

- **Output Layer:** the last layer, the dimension and activation function of which depend on the problem at hand – for example, in binary classification problems it can be a single neuron with a sigmoid activation function.

1.3 Multilayer Perceptron

Next we can proceed to the formal description of the architecture of the whole MLP neural network. In this model we sequentially combine the types of layers discussed prior. We begin the construction of the network from the input layer, then proceed to any positive number of connected hidden layers, and lastly connect the final hidden layer with the output layer. In other words we perform a function composition where the output vector of one layer becomes the input layer of the next one.

Definition 5. *Let $K \in \mathbb{N}_+$ be the number of layers in the neural network, n_0 the dimension of the input vector, and $n_1, n_2, ..., n_K$ the dimensions of the output vectors of subsequent layers in the neural network. A multilayer perceptron (MLP) is defined as the function $F : \mathbb{R}^{n_0} \to \mathbb{R}^{n_K}$ given by the formula:*

$$F(x, \theta) = (f_K \circ f_{K-1} \circ ... \circ f_1)(x),$$

where $f_k : \mathbb{R}^{n_{k-1}} \to \mathbb{R}^{k_j}$ denotes the k-th layer in the network, and $\theta = \{W_1, b_1, ..., W_K, b_K\}$ is the set of all corresponding parameters of those layers (weight matrices W and bias vectors b).

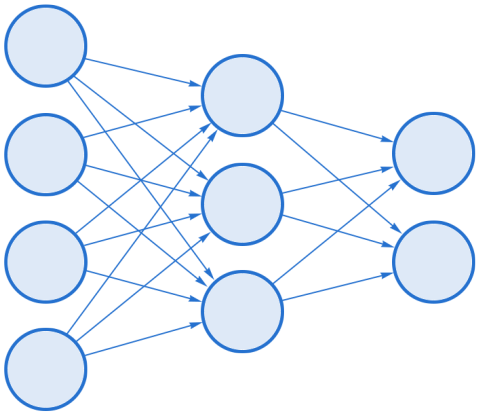

Figure 1: Diagram of a neural network with a single hidden layer.

Figure 1 illustrates one possible function composition.

This way we obtain a model that, with at least one hidden layer using a non-linear activation function, is capable of approximating almost every mathematical function. We can use this model to solve various prediction and classification problems, among many others. The next section is devoted to the process of training the multilayer perceptron, namely the optimization of its parameters.

2 The Problem of Optimization

2.1 The Goal of Training a Neural Network

For a neural network to learn, it must know how far away its output vector is from the real/expected prediction. For this we must define the loss function. This function will look different depending on the type of neural network and the problem being solved. We will introduce a general definition:

Definition 6. *Let $F(x, \theta) \in \mathbb{R}^{n_K}$ be the output of a multilayer perceptron for an input vector $x \in \mathbb{R}^m$ and let $y \in \mathcal{U}$ be the target value from a space $\mathcal{U}$, which corresponds to the input vector. A loss function is defined as a continuous, differentiable almost everywhere function $\mathcal{L} : \mathbb{R}^{n_K} \times \mathcal{U} \to [0, \infty)$.*

The loss function measures the prediction error of a single observation. In "supervised" models, a vector of target values y is given alongside the observations, whereas in "unsupervised" models, these values do not exist. Instead, they are derived from the input vector.

The type of problem determines the loss function and the space of target values $\mathcal{U}$. In classification problems, the "Cross Entropy" loss function is often used, whereas mean squared error is often used in regression problems.

With the help of this function we are able to evaluate the error for a single sample of data. However, to evaluate model error over the entire dataset, we must define the cost function:

Definition 7. *Let $\mathcal{T} = \{(x_1, y_1), (x_2, y_2), ..., (x_N, y_N)\}$ be a dataset with N samples, and let Θ be the space of all possible multilayer perceptron parameters. Then, the cost function is defined as $J : \Theta \to [0, \infty)$, given by the formula:*

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(F(x_i, \theta), y_i).$$

It is an average value of all loss functions for the set $\mathcal{T}$. On this basis, we can define the goal of training the neural network as finding the optimal parameter vector $\hat{\theta} \in \Theta$, which minimizes the value of the cost function $J(\Theta)$ for a chosen dataset $\mathcal{T}$:

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmin}}\, J(\theta).$$

- **Training Set:** The dataset $\mathcal{T}$ is commonly referred to as a "training set" – that is, a dataset on which the neural network trains.
- **Test Set:** The "test" set is the dataset on which a "trained" neural network is evaluated on its performance on data it has never "seen" before.

Based on this, we can proceed to the actual process of training the neural network.

2.2 Process of Training a Neural Network

Finding the global minimum of the cost function $J(\theta)$ is in practice extremely difficult. The multilayer structure of the network and the nonlinearity introduced by the activation functions of the hidden layers render the cost function highly complex and multidimensional. For this reason it is impossible to find an analytical solution to the equation $\nabla_\theta J(\theta) = 0$.

To minimize the function $J(\theta)$ we will utilize iterative numerical methods. The most standard optimization algorithm of this type is **gradient descent**. This algorithm uses the gradient vector, which points in the direction of the function's steepest ascent. To minimize the function, we change the model parameters by "moving" along the negative gradient.

Definition 8. *Let $\alpha > 0$ be a hyperparameter known as the **learning rate**, and let $t \in \mathbb{N}$ denote the training iteration. Gradient descent refers to the iterative update of the neural network's parameter vector θ according to the formula:*

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta J(\theta_t),$$

where $\nabla_\theta J(\theta_t)$ is the vector gradient of the cost function with respect to the neural network's parameter vector, evaluated for the parameters from iteration t.

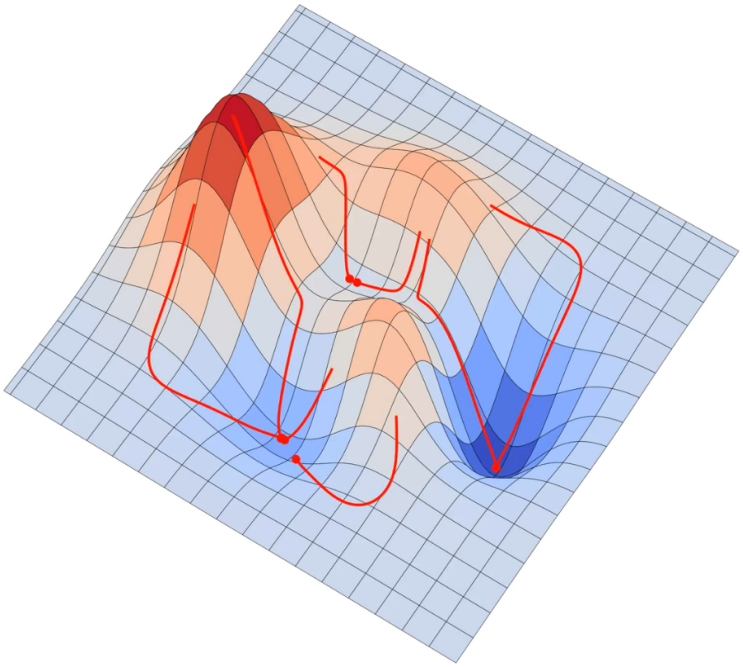

Figure 2: Visualization of gradient descent in 3D space (source: Wikimedia Commons, public domain).

Next it will be necessary to differentiate the activation functions, for that reason it is important to eliminate the problem of non-differentiability.

Remark 2.1 (Subgradient). For activation functions that lack a classical derivative (e.g., at $x = 0$ for the ReLU function), the optimization process employs the so called subgradients. In computing practice, a default value is adopted for problematic points (e.g. 0 for $x = 0$ in the ReLU function), which allows to differentiate and not disrupt the learning process.

Neural networks typically possess thousands or millions of distinct weight and bias parameters, for this reason the naive calculation of derivatives for each parameter individually would be non-feasible due to the computation time. The solution to this is the **backpropagation** algorithm.

2.3 Backpropagation

Backpropagation allows to efficiently compute $\nabla_\theta J(\theta)$. This algorithm relies on the application of the chain rule for the derivatives of composite functions. The previously mentioned "naive" method also utilizes this rule, but not in a backward manner.

Instead of calculating the influence of each weight and bias parameter from the beginning of the entire neural network, the backpropagation algorithm determines the error at the very end of the network and "assigns blame" for that error backwards, layer by layer. This process unfolds as follows:

1. **Forward pass**: During this pass, the input vector data travels through the entire neural network. For each layer k we calculate and store the vector $z_k = W_k x_{k-1} + b_k$ as well as the layer's output vector after activation, $x_k = \sigma(z_k)$. We will use these "signals" in later calculations of gradients.

2. **Backward pass**: During this pass, we utilize the backpropagation algorithm. We introduce the concept of **error signal**, denoted as $\delta_k = \frac{\partial \mathcal{L}}{\partial z_k}$, which represents the "blame" of neurons in layer k for the networks output after the forward pass.

Next we formalize the algorithm of backpropagation.

Definition 9. *Let K denote the number of layers in a neural network. The algorithm of backpropagation determines the parameter gradients as follows:*

1. Determining the error signal at the output layer. *The output layer error signal is the gradient of the loss function multiplied by the derivative of the activation function:*

$$\delta_K = \nabla_{x_K} \mathcal{L} \odot \sigma'(z_K),$$

where $\odot$ denotes the Hadamard product (point-wise vector multiplication).

2. Backpropagating the error signal to the hidden layers. *To determine the error signal of a hidden layer k, each neuron in that layer sums up the error it caused in layer $k+1$:*

$$\delta_k = (W_{k+1}^T \delta_{k+1}) \odot \sigma'(z_k).$$

3. Determining the parameter vector gradient. *Given the calculated error signal values for each neuron in the network, gradients of the parameters for layer k are calculated as follows:*

$$\nabla_{b_k}\mathcal{L} = \delta_k$$

$$\nabla_{W_k}\mathcal{L} = \delta_k x_{k-1}^T.$$

Upon obtaining the loss function gradients for all layers, we relate them to the global cost function $J(\theta)$ to update the network parameters. For a dataset consisting of N samples, global gradients for the weight matrix and the bias parameter vector in layer k are:

$$\nabla_{W_k} J = \frac{1}{N}\sum_{i=1}^{N} \nabla_{W_k}\mathcal{L}^{(i)},$$

$$\nabla_{b_k} J = \frac{1}{N}\sum_{i=1}^{N} \nabla_{b_k}\mathcal{L}^{(i)},$$

where $\mathcal{L}^{(i)}$ is the loss function calculated for the i-th sample.

We recall from Definition 5, that $\theta = \{W_1, b_1, ..., W_K, b_K\}$ is a set of all respective parameters of a multilayer perceptron with K layers. Consequently, the gradient vector of the entire cost function $J(\theta)$ is the set of averaged gradients for each layer:

$$\nabla_\theta J(\theta) = \{\nabla_{W_1} J, \nabla_{b_1} J, ..., \nabla_{W_K} J, \nabla_{b_K} J\}.$$

Using the vector formulated in this way in the algorithm of steepest descent from Definition 8, we update all parameters of the multilayer perceptron.

Remark 2.2 (Types of the gradient descent algorithm). The cost function $J(\theta)$ and its gradients are calculated based on the entire training dataset with N samples. This way a single update of the network parameters is only possible after processing all the samples in the training set (batch gradient descent). In practice, two other methods are also employed:

- **Stochastic Gradient Descent:**

  In each iteration this optimizer updates the parameters after calculating the gradient for just a single sample ($N = 1$). Such updates are

fast, however the direction is rather erratic and often requires more iterations to achieve a minimum.

- **Mini-Batch Gradient Descent:** In each iteration this optimizer randomly selects a small subset of data $\mathcal{P} \subset \mathcal{T}$ of size $M < N$ (e.g. 64 samples) from the training set. The cost function $J_{\mathcal{P}}(\theta)$ and its gradient $\nabla_\theta J_{\mathcal{P}}$ are calculated based on this subset. This approach allows stable model convergence by avoiding updating the parameters after every sample, and also resolves memory issues caused by processing large datasets at once.

3 Problem of Overfitting

The goal of training a multilayer perceptron is not to memorize the data from the training set. We need the model to achieve the capability for generalization – that is, to perform well on data it has never seen. When a neural network is trained for too many iterations or possesses excessive capacity relative to the data, the phenomenon of **overfitting** may occur.

Overfitting occurs when the value of the cost function $J(\theta)$ decreases on the training set while the error on the test set increases. Various regularization techniques are used to prevent this memorization of noise present in the training samples. We will focus on one of the most widely used methods, known as **dropout**.

3.1 The Dropout Regularization Technique

Dropout prevents a multilayer perceptron from becoming overly dependent on any single neuron. Another issue it resolves is **co-adaptation**. It occurs when neurons rely on the output values of other neurons rather than learning independently.

Definition 10. *Let $p \in (0, 1)$ be the hyperparameter for the probability of "dropping out" a neuron.* ***Dropout regularization*** *for a hidden layer k is then referred to as the modification of its output vector $\sigma(z_k)$ during each forward pass propagation step. To achieve this, a vector m_k is generated for each hidden layer k, where the elements are independent random variables drawn from the Bernoulli distribution:*

$$m_k \sim \text{Bernoulli}(1 - p).$$

The input vector x of the layer $k + 1$ is modified by applying the vector m_k using the Hadamard product:

$$\tilde{x}_{k+1} = m_k \ \odot \ \sigma(z_k)$$

In this way, a different reduced sub-network is sampled during forward propagation for each individual sample. This forces greater independence for neurons, which improves the multilayer perceptron's capability for generalization.

Bibliography

[1] M. A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, San Francisco, 2015. *http://neuralnetworksanddeeplearning.com*. Accessed 27-05-2026.

[2] H. Kinsley, D. Kukieła. *Neural Networks from Scratch in Python*. NNFS, USA, 2020.

[3] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, Cambridge, 2016.